

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Базы данных

ОТЧЕТ

Тема «ЖД-вокзал»

по лабораторной работе № 1

на тему

РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ ПРИКЛАДНОЙ ПРОГРАММЫ

Студент:

Д. А. Снитко

Преподаватель:

С.С. Силич

МИНСК 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ	4
1.1 Описание реляционной модели	4
1.2 Описание таблиц	4
1.3 Выделение справочных таблиц	6
1.4 Выделение прав доступа	6
1.5 Разработка серверной части.....	6
2 ПРОГРАММИРОВАНИЕ СЕРВЕРНОЙ ЧАСТИ	8
2.1 Создание скриптов	8
2.2 Реализация HTTP-сервера.....	10
2.3 Тестирование HTTP-сервера.....	11
ЗАКЛЮЧЕНИЕ	18
ПРИЛОЖЕНИЕ А	19

ВВЕДЕНИЕ

Данная работа посвящена комплексной разработке серверной части прикладной системы, работающей с реляционной базой данных Railway. Целью является формализация технических требований, определение ролевой модели доступа, проектирование и реализация REST-ориентированного HTTP-сервера для безопасного и удобного взаимодействия с данными.

Предметная область включает управление поездками и инфраструктурой железнодорожных перевозок: справочные сведения о маршрутах, станциях, поездах и сотрудниках, а также операционные таблицы, отражающие билеты, расписание и связи между объектами. Частью работы является корректное разграничение ответственности и прав доступа: обычный пользователь взаимодействует с основными (операционными) таблицами, выполняя просмотр и редактирование транзакционных данных, а суперпользователь, помимо этих действий, имеет право изменять справочные сущности и осуществлять административные операции резервного копирования и восстановления базы данных.

Серверная часть реализуется как HTTP-сервис с определёнными ресурсами и стандартными методами взаимодействия (GET, POST, PUT, DELETE). Обмен данными выполняется в формате JSON, что обеспечивает совместимость с широким набором клиентских приложений и упрощает интеграцию.

Нефункциональные требования включают безопасность (Basic-аутентификация, проверка ролей и ограничение доступа к справочным ресурсам), производительность (рациональные SQL-операции, минимизация лишних обращений), и надёжность (бэкапы и возможность полной реконструкции базы из резервной копии). Поддерживаются как системные бэкапы в форме SQL-дампов с использованием инструментов PostgreSQL, так и табличные выгрузки в формат Excel для точечного восстановления. Эти механизмы позволяют быстро возвращаться к согласованному состоянию данных, документировать результаты и проводить анализ.

– Категория: категория билета (тариф/класс). Тип varchar(50).

Таблица Маршрут содержит справочные сведения о маршрутах следования. Описание имён таблицы Маршрут:

- id: идентификатор маршрута. Первичный ключ;
- Время_в_пути: длительность пути. Тип interval;
- Количество_остановок: число остановок на маршруте. Тип integer;
- Название: наименование маршрута. Тип varchar;
- Расстояние: протяжённость маршрута. Тип numeric.

Таблица Маршрут_Станция представляет связь маршрута со станциями (many-to-many). Описание имён таблицы Маршрут_Станция:

- id: идентификатор записи связи. Первичный ключ;
- Маршрут_id: внешний ключ на таблицу Маршрут (id);
- Станция_id: внешний ключ на таблицу Станция (id);
- Порядок_остановки: порядковый номер остановки в маршруте. Тип integer.

Таблица Пассажир содержит сведения о пассажирах. Описание имён таблицы Пассажир:

- id: идентификатор пассажира. Первичный ключ;
- ФИО: полное имя пассажира. Тип varchar(255), NOT NULL;
- Номер_паспорта: номер паспорта. Тип varchar(50), NOT NULL, уникальное значение;
- Пол: пол пассажира. Тип varchar(10);
- Контактный_телефон: телефон для связи. Тип varchar(20), NOT NULL.

Таблица Поезд содержит сведения о поездах. Описание имён таблицы Поезд:

- id: идентификатор поезда. Первичный ключ;
- Количество_вагонов: число вагонов в составе. Тип integer;
- Вместимость: суммарная вместимость поезда. Тип integer;
- Компания: перевозчик/оператор. Тип varchar(100), NOT NULL;
- Тип: тип поезда (например, скорый, пассажирский). Тип varchar(50).

Таблица Поезд_Сотрудник представляет связь поезда с персоналом (many-to-many). Описание имён таблицы Поезд_Сотрудник:

- id: идентификатор записи связи. Первичный ключ;
- Поезд_id: внешний ключ на таблицу Поезд (id);
- Сотрудник_id: внешний ключ на таблицу Сотрудник (id).

Таблица Расписание содержит сведения о рейсах и времени движения. Описание имён таблицы Расписание:

- id: идентификатор записи расписания. Первичный ключ;
- Поезд_id: внешний ключ на таблицу Поезд (id);
- Маршрут_id: внешний ключ на таблицу Маршрут (id);
- Станция_отбытия_id: внеш. ключ на таблицу Станция (id) — станция отправления;

– Станция_прибытия_id: внеш. ключ на таблицу Станция (id) — станция прибытия;

– Время_отбытия: время отправления. Тип time, NOT NULL;

– Время_прибытия: время прибытия. Тип time, NOT NULL.

Таблица Сотрудник содержит сведения о персонале. Описание имён таблицы Сотрудник:

– id: идентификатор сотрудника. Первичный ключ;

– ФИО: полное имя сотрудника. Тип varchar(255), NOT NULL;

– Контактный_телефон: телефон для связи. Тип varchar(20), NOT NULL;

– Должность: должность сотрудника. Тип varchar(100), NOT NULL;

– Стаж: стаж работы. Тип integer.

Таблица Сотрудник_Маршрут представляет связь сотрудника с маршрутом (many-to-many). Описание имён таблицы Сотрудник_Маршрут:

– id: идентификатор записи связи. Первичный ключ;

– Сотрудник_id: внешний ключ на таблицу Сотрудник (id);

– Маршрут_id: внешний ключ на таблицу Маршрут (id).

Таблица Станция содержит справочные сведения о станциях. Описание имён таблицы Станция:

– id: идентификатор станции. Первичный ключ;

– Название: название станции. Тип varchar(100), NOT NULL;

– Город: город расположения. Тип varchar(100), NOT NULL;

– Количество_платформ: число платформ. Тип integer;

– Количество_путей: число путей. Тип integer.

1.3 Выделение справочных таблиц

К справочным относятся Маршрут, Станция, Поезд и Сотрудник, так как они описывают внутреннюю структуру и поддерживаются администратором.

1.4 Выделение прав доступа

Обычный пользователь обладает правами просмотра и редактирования основных таблиц. Суперпользователь имеет те же права и дополнительно может редактировать справочные таблицы, а также выполнять операции резервного копирования и восстановления БД. Действия, требующие повышенных прав, защищены паролем суперпользователя.

1.5 Разработка серверной части

Серверная часть прикладной программы реализована в виде HTTP-сервера и использует формат JSON для тел запросов и ответов. Для взаимодействия с ресурсами применяются стандартные HTTP-методы:

1) GET – получение данных о ресурсе;

- 2) POST – создание нового ресурса;
- 3) PUT – обновление существующего ресурса;
- 4) DELETE – удаление ресурса.

Каждая операция доступна по уникальным URL:

- /tables – список всех таблиц;
- /tables/{table} – чтение содержимого таблицы;
- /tables/{table}/schema – получение схемы столбцов;
- /tables/{table} [POST] – добавление записей;
- /tables/{table}/rows/{id} [PUT] – обновление записи;
- /tables/{table}/rows/{id} [DELETE] – удаление записи;
- /tables/{table}/create [POST] – создание таблицы;
- /tables/{table}/delete [DELETE] – удаление таблицы;
- /tables/{table}/backup [POST] – копирование таблицы в Excel;
- /tables/{table}/restore [POST] – восстановление таблицы из Excel;
- /database/backup [POST] – резервное копирование всей БД в SQL;
- /database/restore [POST] – восстановление БД из SQL;
- /query [POST] – выполнение произвольных SQL-запросов;
- /queries [GET/POST] – получение и сохранение запросов.

Серверная часть прикладной программы должна предоставлять следующие операции для работы с базой данных:

- просмотр таблиц;
- фильтрация содержимого таблиц;
- добавление записей в таблицы;
- обновление записей в таблицах;
- удаление записей из таблиц;
- выполнение специальных запросов;
- создание бэкапов базы данных;
- сохранение результатов запросов в файл.

2 ПРОГРАММИРОВАНИЕ СЕРВЕРНОЙ ЧАСТИ

2.1 Создание скриптов

Для создания таблиц в базе данных используется следующий скрипт:

```
CREATE TABLE IF NOT EXISTS public."Билет"
(
    id serial NOT NULL,
    "Пассажир_id" integer NOT NULL,
    "Поезд_id" integer NOT NULL,
    "Маршрут_id" integer NOT NULL,
    "Место" integer NOT NULL,
    "Время_покупки" timestamp without time zone NOT NULL
    DEFAULT now(),
    "Стоимость" numeric(10, 2),
    "Категория" character varying(50) COLLATE
pg_catalog."default",
    CONSTRAINT "Билет_pkey" PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public."Маршрут"
(
    id serial NOT NULL,
    "Время_в_пути" interval NOT NULL,
    "Количество_остановок" integer,
    "Название" character varying NOT NULL,
    "Расстояние" numeric NOT NULL,
    CONSTRAINT "Маршрут_pkey" PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public."Маршрут_Станция"
(
    id serial NOT NULL,
    "Маршрут_id" integer NOT NULL,
    "Станция_id" integer NOT NULL,
    "Порядок_остановки" integer NOT NULL,
    CONSTRAINT "Маршрут_Станция_pkey" PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public."Пассажир"
(
    id serial NOT NULL,
    "ФИО" character varying(255) COLLATE pg_catalog."default"
    NOT NULL,
    "Номер_паспорта" character varying(50) COLLATE
pg_catalog."default" NOT NULL,
    "Пол" character varying(10) COLLATE pg_catalog."default",
    "Контактный_телефон" character varying(20) COLLATE
pg_catalog."default" NOT NULL,
    CONSTRAINT "Пассажир_pkey" PRIMARY KEY (id),
```



```

        CONSTRAINT "Пассажир_Номер_паспорта_key" UNIQUE
("Номер_паспорта")
    );

CREATE TABLE IF NOT EXISTS public."Поезд"
(
    id serial NOT NULL,
    "Количество_вагонов" integer,
    "Вместимость" integer,
    "Компания" character varying(100) COLLATE
pg_catalog."default" NOT NULL,
    "Тип" character varying(50) COLLATE pg_catalog."default",
    CONSTRAINT "Поезд_pkey" PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public."Поезд_Сотрудник"
(
    id serial NOT NULL,
    "Поезд_id" integer NOT NULL,
    "Сотрудник_id" integer NOT NULL,
    CONSTRAINT "Поезд_Сотрудник_pkey" PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public."Расписание"
(
    id serial NOT NULL,
    "Поезд_id" integer NOT NULL,
    "Маршрут_id" integer NOT NULL,
    "Станция_отбытия_id" integer NOT NULL,
    "Станция_прибытия_id" integer NOT NULL,
    "Время_отбытия" time without time zone NOT NULL,
    "Время_прибытия" time without time zone NOT NULL,
    CONSTRAINT "Расписание_pkey" PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public."Сотрудник"
(
    id serial NOT NULL,
    "ФИО" character varying(255) COLLATE pg_catalog."default"
NOT NULL,
    "Контактный_телефон" character varying(20) COLLATE
pg_catalog."default" NOT NULL,
    "Должность" character varying(100) COLLATE
pg_catalog."default" NOT NULL,
    "Стаж" integer,
    CONSTRAINT "Сотрудник_pkey" PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public."Сотрудник_Маршрут"
(
    id serial NOT NULL,
    "Сотрудник_id" integer NOT NULL,

```

```

        "Маршрут_id" integer NOT NULL,
        CONSTRAINT "Сотрудник_Маршрут_pkey" PRIMARY KEY (id)
    );

CREATE TABLE IF NOT EXISTS public."Станция"
(
    id serial NOT NULL,
    "Название" character_varying(100) COLLATE
pg_catalog."default" NOT NULL,
    "Город" character_varying(100) COLLATE
pg_catalog."default" NOT NULL,
    "Количество_платформ" integer,
    "Количество_путей" integer,
    CONSTRAINT "Станция_pkey" PRIMARY KEY (id)
);

```

2.2 Реализация HTTP-сервера

Для создания серверной части использован стек FastAPI + psycorg2 с запуском через uvicorn. Описаны модели данных для тел запросов и реализованы универсальные контроллеры для работы с таблицами БД Railway. Основные функции программы:

1. Подключение к БД:
 - Соединение с PostgreSQL (БД Railway) через psycorg2;
 - Нормализация типов для создания таблиц.
2. Управление бэкапами:
 - /database/backup – создание резервной копии БД через pg_dump;
 - /database/restore – восстановление БД через psql;
 - Использование переменной среды PGPASSWORD для авторизации;
 - /tables/{table}/backup – экспорт данных таблицы в Excel;
 - /tables/{table}/restore – восстановление из Excel.
3. Базовые CRUD-операции:
 - Добавление, обновление и удаление записей в заданной таблице;
 - Получение содержимого и схемы таблиц.
4. REST API сервер:
 - GET/POST/PUT/DELETE эндпоинты, соответствующие операциям над таблицами;
 - Дополнительные эндпоинты для произвольных SQL и управления сохранёнными запросами.
5. Система безопасности:
 - Ролевая модель с проверкой прав;
 - Ограничение редактирования справочных таблиц для обычного пользователя.

Сервер запускается на порту 8000 и предоставляет полный API для управления базой данных Railway.

2.3 Тестирование HTTP-сервера

Для тестирования использован Swagger UI, доступный по адресу /docs. Через интерфейс проверены GET/POST/PUT/DELETE операции для основных эндпоинтов, а также создание и восстановление резервных копий. Валидация входных данных выполняется средствами FastAPI/Pydantic.

На рисунках 2.1 и 2.2 отображены результаты выполнения запросов чтения содержимого всех таблиц и таблицы Поезд соответственно.

Server response	
Code	Details
200	Response body <pre>["Станция", "Маршрут", "Билет", "Пассажир", "Поезд", "Маршрут_Станция", "Поезд_Сотрудник", "Сотрудник", "Расписание", "Сотрудник_Маршрут"]</pre>

Рисунок 2.1 – Результат запроса таблицы Пассажир (GET /tables)

Name	Description
table_name required	Поезд
string (path)	
<button>Execute</button>	
Responses	
Curl	
<pre>curl -X 'GET' \ 'http://localhost:8000/tables/%D0%9F%D0%B5%D0%B7%D0%B4' \ -H 'accept: application/json'</pre>	
Request URL	
<pre>http://localhost:8000/tables/%D0%9F%D0%B5%D0%B7%D0%B4</pre>	
Server response	
Code	Details
200	Response body <pre>[{ "id": 1, "Количество вагонов": 20, "Вместимость": 800, "Компания": "РЖД", "Тип": "Скоростной" }, { "id": 2, "Количество вагонов": 15, "Вместимость": 600, "Компания": "РЖД", "Тип": "Пассажирский" }, { "id": 3, "Количество вагонов": 12, "Вместимость": 480, "Компания": "РЖД", "Тип": "Скоростной" }]</pre>

Рисунок 2.2 – Результат запроса таблицы Поезд (GET /tables/Поезд)

На рисунке 2.4 показано получение записей для таблицы Билет, со стоимостью билетов более 70 рублей.

POST /query Execute a custom SQL query

Parameters

No parameters

Request body ^{required}

application/json

Edit Value | Schema

```
{
  "query": "SELECT * FROM \"Билет\" WHERE \"Стоимость\" > 70"
}
```

Рисунок 2.3 – Запрос билетов со стоимостью билетов более 70 рублей

```
curl -X 'POST' \
  'http://localhost:8000/query' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "query": "SELECT * FROM \"Билет\" WHERE \"Стоимость\" > 70"
  }'
```

Request URL

http://localhost:8000/query

Server response

Code	Details
200	Response body <pre>{ "Маршрут_id": 2, "Место": 12, "Время_покупки": "2025-02-20T10:30:00", "Стоимость": 80.5, "Категория": "Эконом" }, { "id": 2, "Пассажир_id": 2, "Поезд_id": 5, "Маршрут_id": 4, "Место": 8, "Время_покупки": "2025-02-18T14:15:00", "Стоимость": 155, "Категория": "Бизнес" }, { "id": 3, "Пассажир_id": 3, "Поезд_id": 7, "Маршрут_id": 1, "Место": 22, "Время_покупки": "2025-02-19T09:00:00", "Стоимость": 100, "Категория": "Эконом" }, { "id": 4, "Пассажир_id": 4, "Поезд_id": 2, "Маршрут_id": 1, "Место": 1, "Время_покупки": "2025-02-19T09:00:00", "Стоимость": 100, "Категория": "Эконом" }</pre>

Рисунок 2.4 – Результат билетов со стоимостью билетов более 70 рублей

На рисунке 2.5 приведён пример создания новой записи в таблице Билет.

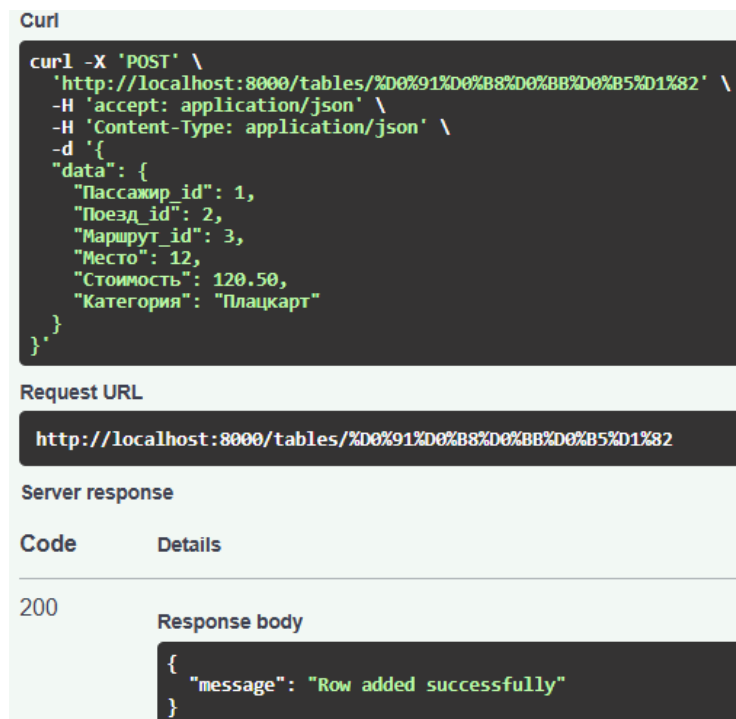


Рисунок 2.5 – Результат POST запроса на добавление записи (POST /tables/Билет)

На рисунке 2.6 показано обновление ранее созданной записи в таблице Билет.

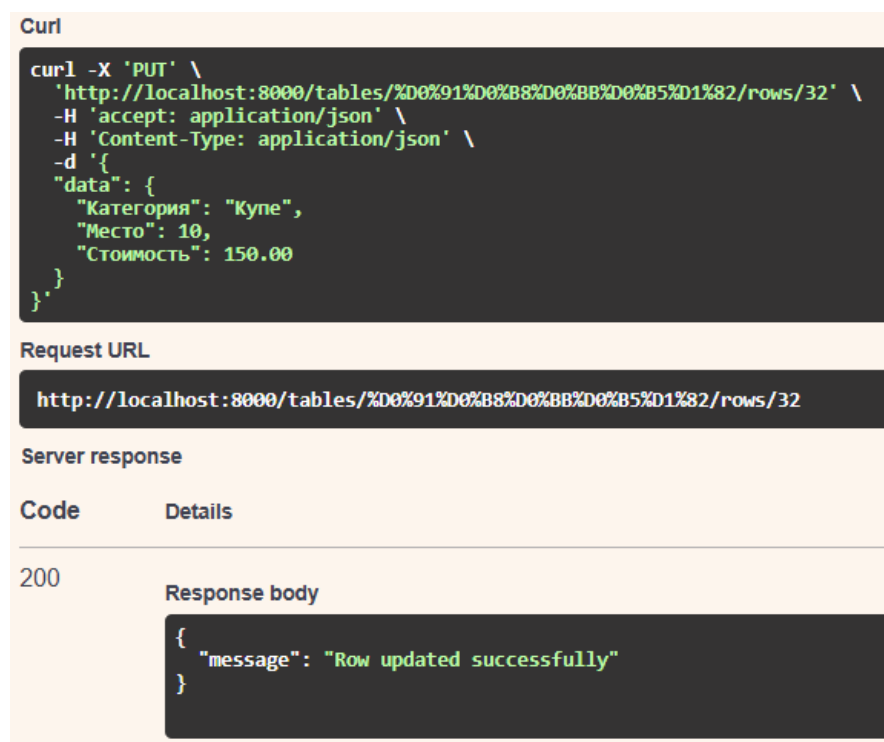


Рисунок 2.6 – Результат PUT запроса на обновление записи (PUT /tables/Билет/rows/{id})

На рисунке 2.7 отображён результат удаления строки из таблицы Билет.



Рисунок 2.7 – Результат DELETE запроса на удаление строки (DELETE /tables/Билет/rows/{id})

На рисунке 2.8 продемонстрирована фильтрация по городу Москва в таблице Станция с использованием SQL-запроса.

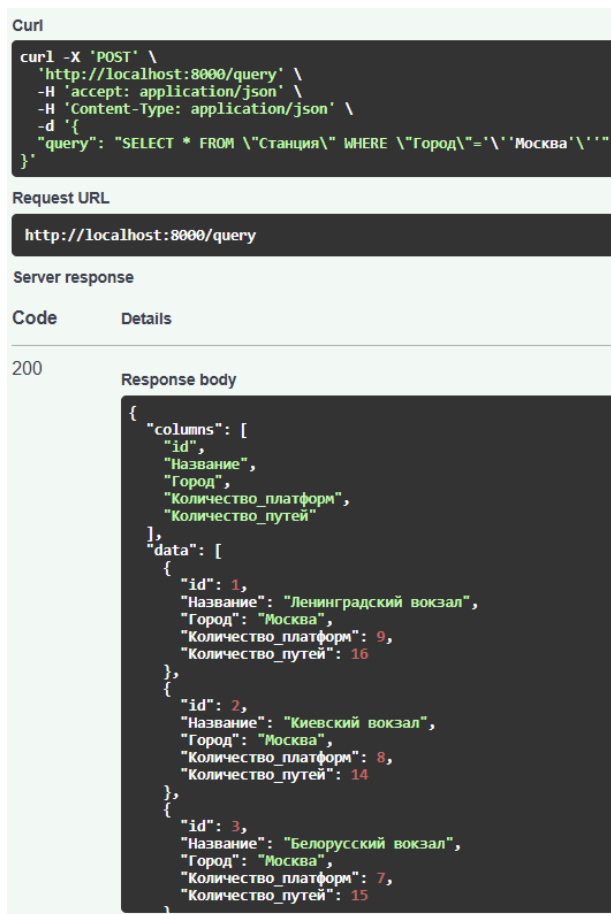


Рисунок 2.8 – Фильтрация станций по городу (POST /query: SELECT * FROM "Станция" WHERE "Город"='Москва')

На рисунке 2.9 показана фильтрация билетов по категории Плацкарт.

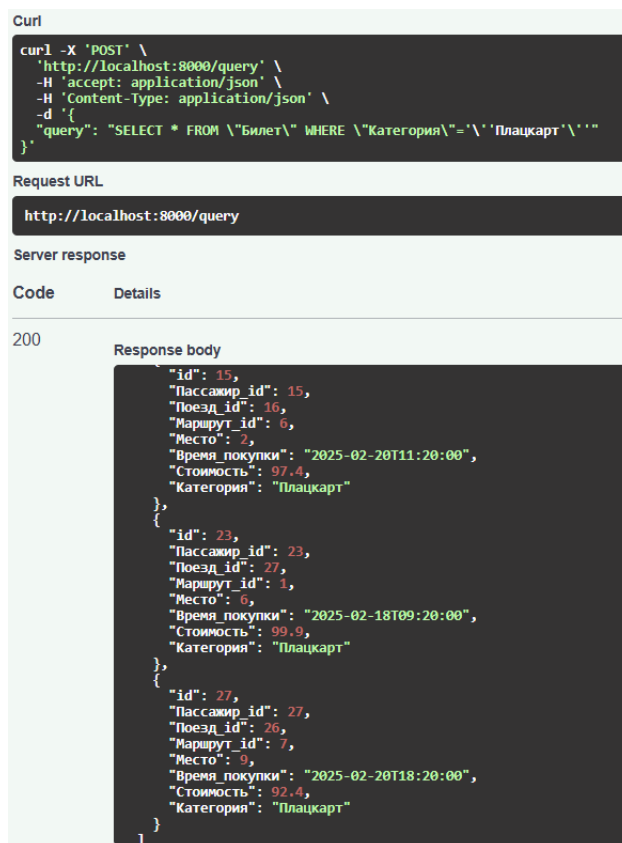


Рисунок 2.9 – Фильтрация билетов по категории (POST /query: SELECT * FROM "Билет" WHERE "Категория"='Плацкарт')

На рисунке 2.10 показано резервное копирование таблицы Билет в Excel.

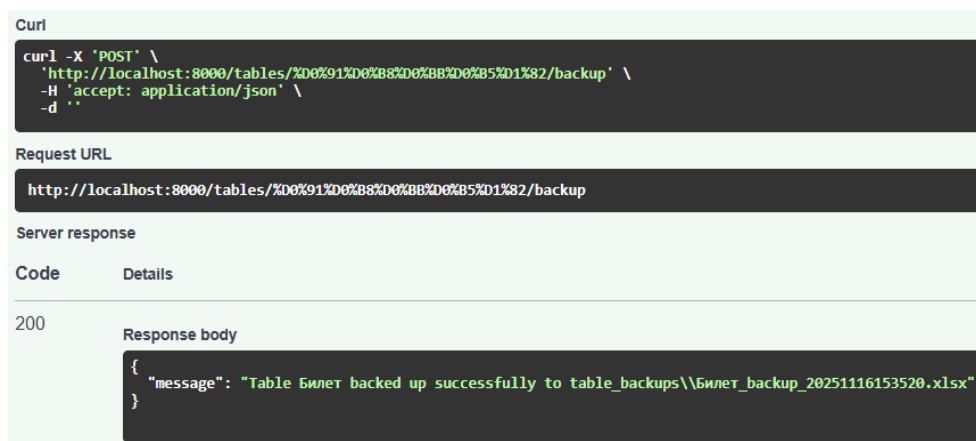


Рисунок 2.10 – Результат создания бэкапа таблицы (POST /tables/Билет/backup)

На рисунке 2.11 приведён пример фильтрации расписания по времени отправления.

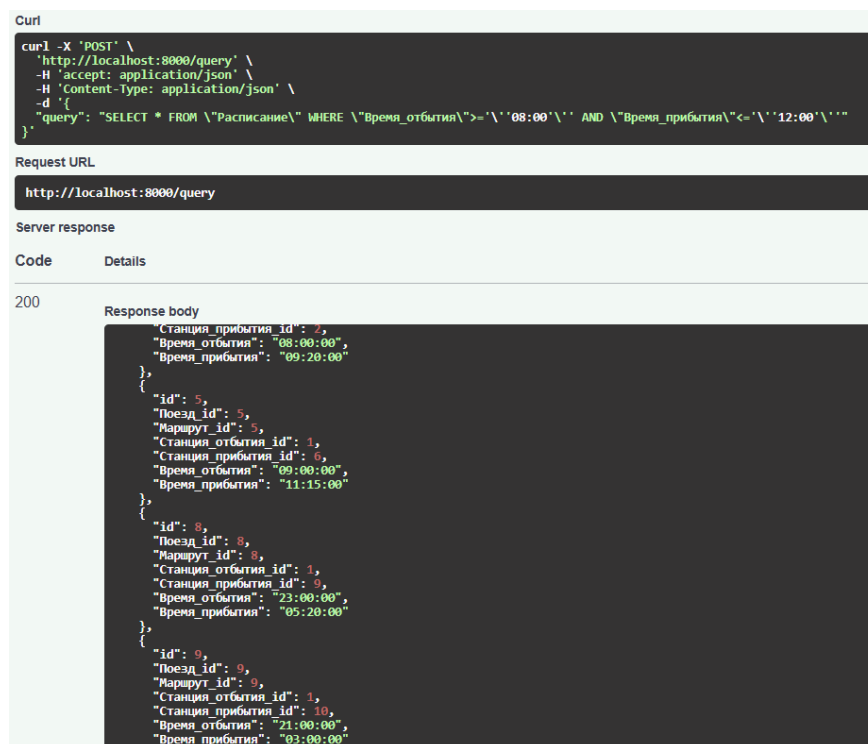


Рисунок 2.11 – Фильтрация расписания по времени (POST /query: SELECT * FROM "Расписание" WHERE "Время_отбытия">='08:00' AND "Время_прибытия"<='12:00')

На рисунке 2.12 отображён отказ в восстановлении таблицы Поезд без прав суперпользователя.

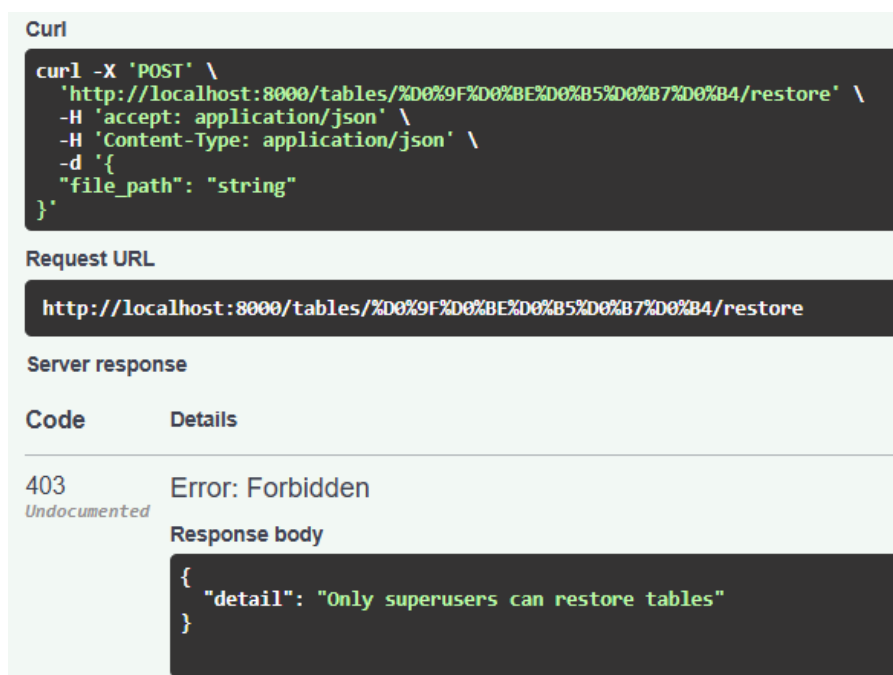
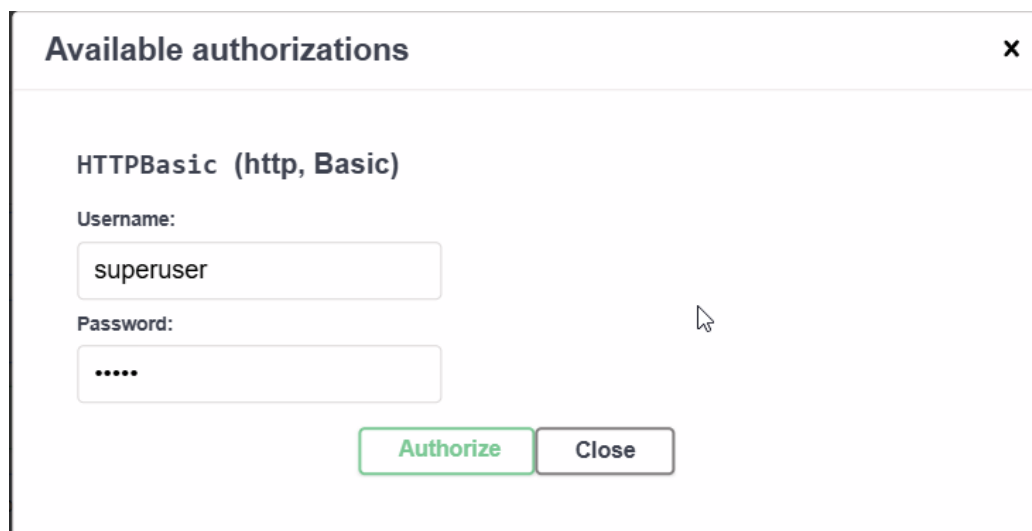


Рисунок 2.12 – Попытка восстановления таблицы без прав суперпользователя (POST /tables/Поезд/restore)

На рисунке 2.13 отображен вход суперпользователя.



Available authorizations ✕

HTTPBasic (http, Basic)

Username:

Password:

Рисунок 2.13 – Вход суперпользователя

На рисунке 2.14 показано успешное восстановление таблицы Поезд с правами суперпользователя.



Curl

```
curl -X 'POST' \
  'http://localhost:8000/tables/%D0%9F%D0%BE%D0%B5%D0%B7%D0%B4/restore' \
  -H 'accept: application/json' \
  -H 'Authorization: Basic c3VwZXJ1c2VyOnN1cGVy' \
  -H 'Content-Type: application/json' \
  -d '{
    "file_path": "table_backups/Поезд_backup_20251116153604.xlsx"
  }'
```

Request URL

http://localhost:8000/tables/%D0%9F%D0%BE%D0%B5%D0%B7%D0%B4/restore

Server response

Code	Details
200	Response body <pre>{ "message": "Table Поезд restored successfully from table_backups/Поезд_backup_20251116153604.xlsx" }</pre>

Рисунок 2.14 – Восстановление таблицы с правами суперпользователя (POST /tables/Поезд/restore)

ЗАКЛЮЧЕНИЕ

В ходе работы реализована серверная часть для БД Railway с чётким разделением ролей доступа, поддержкой стандартных HTTP-методов и использованием формата JSON для обмена данными. Обеспечены операции просмотра, редактирования и администрирования, включая резервное копирование и восстановление, а тестирование выполнено через Swagger UI, что подтверждает корректность работы API.

ПРИЛОЖЕНИЕ А

(обязательное)

Код программы

Файл config.py

```
# Database connection settings
DB_CONFIG = {
    'host': 'localhost',
    'port': 5432,
    'database': 'Railway',
    'user': 'postgres',
    'password': '1111'
}

# Superuser password
SUPERUSER_PASSWORD = 'super'
# List of lookup tables that can only be edited by a superuser
LOOKUP_TABLES = [
    "Маршрут",
    "Станция",
    "Поезд",
    "Сотрудник"
]
```

Файл backend.py

```
import psycopg2
from fastapi import FastAPI, HTTPException, Depends, status, Response
from fastapi.security import HTTPBasic, HTTPBasicCredentials
from pydantic import BaseModel
import uvicorn
from typing import List, Dict, Any
import os
import subprocess
import datetime

from config import DB_CONFIG, SUPERUSER_PASSWORD, LOOKUP_TABLES

# Type normalization for user-friendly inputs
TYPE_ALIASES = {
    'int': 'INTEGER',
    'integer': 'INTEGER',
    'bigint': 'BIGINT',
    'smallint': 'SMALLINT',
    'serial': 'SERIAL',
    'serial4': 'SERIAL',
}
```

```

    'serial8': 'BIGSERIAL',
    'bool': 'BOOLEAN',
    'boolean': 'BOOLEAN',
    'text': 'TEXT',
    'varchar': 'VARCHAR',
    'char': 'CHAR',
    'date': 'DATE',
    'time': 'TIME',
    'timestamp': 'TIMESTAMP',
    'numeric': 'NUMERIC',
    'decimal': 'DECIMAL',
    'float': 'DOUBLE PRECISION',
    'double': 'DOUBLE PRECISION',
    'real': 'REAL'
}

def normalize_type(typ: str) -> str:
    t = typ.strip()
    tl = t.lower()
    # Preserve parameterized types like varchar(100),
    numeric(10,2)
    if '(' in t and ')' in t:
        base = tl.split('(')[0]
        if base in TYPE_ALIASES and base not in ['varchar',
'char', 'numeric', 'decimal']:
            # For fixed alias types with params we ignore params
            return TYPE_ALIASES[base]
            # Force base name to proper case but keep params
        if base in ['varchar', 'char', 'numeric', 'decimal']:
            proper = base.upper()
            return proper + t[t.find('('):]
        return t
    # Non-parameterized alias
    if tl in TYPE_ALIASES:
        return TYPE_ALIASES[tl]
    return t

app = FastAPI(
    title="Railway DB API",
    description="API for managing the railway database, with
role-based access control.",
    version="1.0.0"
)

security = HTTPBasic()
security_optional = HTTPBasic(auto_error=False)

def get_db_connection():
    """Establishes a connection to the database."""
    try:
        conn = psycopg2.connect(**DB_CONFIG)
        yield conn

```

```

        finally:
            conn.close()

class User(BaseModel):
    username: str

def get_current_user(credentials: HTTPBasicCredentials =
Depends(security)):
    if credentials.username == 'user' and credentials.password
== 'user':
        return {"role": "user"}
    elif credentials.username == 'superuser' and
credentials.password == SUPERUSER_PASSWORD:
        return {"role": "superuser"}
    else:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Basic"},
        )

# Optional auth for read-only endpoints

def get_user_or_guest(credentials: HTTPBasicCredentials | None =
Depends(security_optional)):
    if credentials is None:
        return {"role": "user"}
    if credentials.username == 'user' and credentials.password
== 'user':
        return {"role": "user"}
    elif credentials.username == 'superuser' and
credentials.password == SUPERUSER_PASSWORD:
        return {"role": "superuser"}
    else:
        # If provided but incorrect, treat as unauthenticated
        user for read-only
        return {"role": "user"}

@app.get("/tables", response_model=List[str], summary="Get a
list of all tables")
def get_tables(conn=Depends(get_db_connection), user: dict =
Depends(get_user_or_guest)):
    """Retrieves a list of all tables in the public schema."""
    try:
        with conn.cursor() as cursor:
            cursor.execute("""
                SELECT table_name FROM information_schema.tables
                WHERE table_schema = 'public'
            """)
            tables = [table[0] for table in cursor.fetchall()]
            return tables
    except Exception as e:

```

```

        raise HTTPException(status_code=500, detail=str(e))

@app.get("/tables/{table_name}", response_model=List[Dict[str, Any]], summary="Get table content")
def get_table_content(table_name: str,
conn=Depends(get_db_connection), user: dict =
Depends(get_user_or_guest)):
    """Retrieves the content of a specific table."""
    try:
        with conn.cursor() as cursor:
            # Sanitize table_name to prevent SQL injection
            cursor.execute("SELECT table_name FROM
information_schema.tables WHERE table_name = %s", (table_name,))
            if cursor.fetchone() is None:
                raise HTTPException(status_code=404,
detail="Table not found")

            cursor.execute(f'SELECT * FROM "{table_name}"')
            columns = [desc[0] for desc in cursor.description]
            rows = cursor.fetchall()

            data = []
            for row in rows:
                row_dict = {}
                for i, value in enumerate(row):
                    col_name = columns[i]
                    if isinstance(value, datetime.timedelta):
                        total_seconds =
int(value.total_seconds())
                        hours, remainder = divmod(total_seconds,
3600)
                        minutes, seconds = divmod(remainder, 60)
                        row_dict[col_name] =
f"{hours:02}:{minutes:02}:{seconds:02}"
                    elif isinstance(value, datetime.time):
                        row_dict[col_name] =
value.strftime('%H:%M:%S')
                    else:
                        row_dict[col_name] = value
                data.append(row_dict)
            return data
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/tables/{table_name}/schema",
response_model=List[Dict[str, Any]], summary="Get table schema")
def get_table_schema(table_name: str,
conn=Depends(get_db_connection), user: dict =
Depends(get_user_or_guest)):
    try:
        with conn.cursor() as cursor:

```

```

        cursor.execute("SELECT table_name FROM
information_schema.tables WHERE table_name = %s", (table_name,))
        if cursor.fetchone() is None:
            raise HTTPException(status_code=404,
detail="Table not found")
        cursor.execute("""
            SELECT column_name, data_type
            FROM information_schema.columns
            WHERE table_schema='public' AND table_name=%s
            ORDER BY ordinal_position
        """, (table_name,))
        rows = cursor.fetchall()
        return [{"name": r[0], "type": r[1]} for r in rows]
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

class Row(BaseModel):
    data: Dict[str, Any]

class ColumnDefinition(BaseModel):
    name: str
    type: str

class ColumnInfo(BaseModel):
    name: str
    type: str

class CreateTableRequest(BaseModel):
    columns: List[ColumnDefinition]

class TableRestoreRequest(BaseModel):
    file_path: str

class DatabaseRestoreRequest(BaseModel):
    backup_file: str

class QuerySave(BaseModel):
    query: str

@app.post("/tables/{table_name}", summary="Add a new row to a
table")
def add_row(table_name: str, row: Row,
conn=Depends(get_db_connection), user: dict =
Depends(get_current_user)):
    if f'"{table_name}"' in LOOKUP_TABLES and user['role'] !=
'superuser':
        raise HTTPException(status_code=403, detail="User cannot
edit lookup tables")
    try:
        with conn.cursor() as cursor:
            columns = ', '.join(row.data.keys())
            values = ', '.join(['%s'] * len(row.data.values()))

```

```

        query = f'INSERT INTO "{table_name}" ({columns})
VALUES ({values})'
        cursor.execute(query, list(row.data.values()))
        conn.commit()
        return {"message": "Row added successfully"}
    except Exception as e:
        conn.rollback()
        raise HTTPException(status_code=500, detail=str(e))

@app.put("/tables/{table_name}/rows/{row_id}", summary="Update a
row in a table")
def update_row(table_name: str, row_id: int, row: Row,
conn=Depends(get_db_connection), user: dict =
Depends(get_current_user)):
    if f'"{table_name}"' in LOOKUP_TABLES and user['role'] !=
'superuser':
        raise HTTPException(status_code=403, detail="User cannot
edit lookup tables")
    try:
        with conn.cursor() as cursor:
            updates = ', '.join([f"{key} = %s" for key in
row.data.keys()])
            query = f'UPDATE "{table_name}" SET {updates} WHERE
id = %s'
            cursor.execute(query, list(row.data.values()) +
[row_id])
            conn.commit()
            return {"message": "Row updated successfully"}
    except Exception as e:
        conn.rollback()
        raise HTTPException(status_code=500, detail=str(e))

@app.delete("/tables/{table_name}/rows/{row_id}",
summary="Delete a row from a table")
def delete_row(table_name: str, row_id: int,
conn=Depends(get_db_connection), user: dict =
Depends(get_current_user)):
    if f'"{table_name}"' in LOOKUP_TABLES and user['role'] !=
'superuser':
        raise HTTPException(status_code=403, detail="User cannot
edit lookup tables")
    try:
        with conn.cursor() as cursor:
            query = f'DELETE FROM "{table_name}" WHERE id = %s'
            cursor.execute(query, (row_id,))
            conn.commit()
            return {"message": "Row deleted successfully"}
    except Exception as e:
        conn.rollback()
        raise HTTPException(status_code=500, detail=str(e))

class Query(BaseModel):

```



```

        query: str

@app.post("/query", summary="Execute a custom SQL query")
def execute_query(query: Query, conn=Depends(get_db_connection),
user: dict = Depends(get_current_user)):
    try:
        with conn.cursor() as cursor:
            cursor.execute(query.query)
            if cursor.description:
                columns = [desc[0] for desc in
cursor.description]
                rows = cursor.fetchall()

                data = []
                for row in rows:
                    row_dict = {}
                    for i, value in enumerate(row):
                        col_name = columns[i]
                        if isinstance(value,
datetime.timedelta):
                            total_seconds =
int(value.total_seconds())
                            hours, remainder =
divmod(total_seconds, 3600)
                            minutes, seconds = divmod(remainder,
60)
                            row_dict[col_name] =
f"{hours:02}:{minutes:02}:{seconds:02}"
                        elif isinstance(value, datetime.time):
                            row_dict[col_name] =
value.strftime('%H:%M:%S')
                        else:
                            row_dict[col_name] = value
                    data.append(row_dict)
                return {"columns": columns, "data": data}
            else:
                conn.commit()
                return {"message": "Query executed
successfully"}
        except Exception as e:
            conn.rollback()
            raise HTTPException(status_code=500, detail=str(e))

@app.post("/queries/new", summary="Create a new query")
def new_query(user: dict = Depends(get_current_user)):
    # This endpoint is mainly for frontend to clear the query
    text
    return {"message": "New query created"}

@app.post("/queries/run_saved", summary="Run a saved query")

```

```

def run_saved_query(query: Query,
conn=Depends(get_db_connection), user: dict =
Depends(get_current_user)):
    return execute_query(query, conn, user)

@app.post("/tables/{table_name}/create", summary="Create a
table")
def create_table(table_name: str, request: CreateTableRequest,
conn=Depends(get_db_connection), user: dict =
Depends(get_current_user)):
    # Restrict only for lookup tables; allow normal user for
other tables
    if f'"{table_name}"' in LOOKUP_TABLES and user['role'] !=
'superuser':
        raise HTTPException(status_code=403, detail="Only
superusers can create lookup tables")
    try:
        with conn.cursor() as cursor:
            cols = []
            seen = set()
            for c in request.columns:
                name = c.name.strip()
                typ = normalize_type(c.type.strip())
                if not name or not typ:
                    raise HTTPException(status_code=422,
detail="Column name and type are required")
                if name.lower() in seen:
                    raise HTTPException(status_code=422,
detail=f"Duplicate column name: {name}")
                seen.add(name.lower())
                cols.append(ColumnDefinition(name=name,
type=typ))

            has_id = any(col.name.lower() == 'id' for col in
cols)
            if not has_id:
                cols.insert(0, ColumnDefinition(name='id',
type='SERIAL PRIMARY KEY'))

            column_defs = ', '.join([f'"{col.name}" {col.type}'
for col in cols])
            create_query = f'CREATE TABLE "{table_name}"
({column_defs})'
            cursor.execute(create_query)
            # Insert one empty row (NULLs) to match UI
expectations
            non_id_cols = [f'"{col.name}"' for col in cols if
col.name.lower() != 'id']
            if non_id_cols:
                placeholders = ', '.join(['%s'] *
len(non_id_cols))

```

```

        insert_query = f'INSERT INTO "{table_name}" ({",
".join(non_id_cols)}) VALUES ({placeholders})'
        cursor.execute(insert_query, [None] *
len(non_id_cols))
        conn.commit()
        return {"message": f"Table {table_name} created
successfully."}
    except HTTPException:
        raise
    except Exception as e:
        conn.rollback()
        raise HTTPException(status_code=500, detail=str(e))

@app.delete("/tables/{table_name}/delete", summary="Delete a
table")
def delete_table(table_name: str,
conn=Depends(get_db_connection), user: dict =
Depends(get_current_user)):
    # Restrict only lookup tables for superuser
    if f'"{table_name}"' in LOOKUP_TABLES and user['role'] !=
'superuser':
        raise HTTPException(status_code=403, detail="Only
superusers can delete lookup tables")
    try:
        with conn.cursor() as cursor:
            cursor.execute(f'DROP TABLE "{table_name}"')
            conn.commit()
            return {"message": f"Table {table_name} deleted
successfully."}
    except Exception as e:
        conn.rollback()
        raise HTTPException(status_code=500, detail=str(e))

import pandas as pd

@app.post("/tables/{table_name}/backup", summary="Backup a
table")
def backup_table(table_name: str,
conn=Depends(get_db_connection), user: dict =
Depends(get_current_user)):
    # Allow backup for any table; restrict edits elsewhere
    try:
        with conn.cursor() as cursor:
            cursor.execute(f'SELECT * FROM "{table_name}"')
            data = cursor.fetchall()
            columns = [desc[0] for desc in cursor.description]
            df = pd.DataFrame(data, columns=columns)
            backup_dir = "table_backups"
            if not os.path.exists(backup_dir):
                os.makedirs(backup_dir)
            timestamp =
datetime.datetime.now().strftime("%Y%m%d%H%M%S")

```

```

        backup_file = os.path.join(backup_dir,
f'{table_name}_backup_{timestamp}.xlsx')
        df.to_excel(backup_file, index=False)
        return {"message": f"Table {table_name} backed up
successfully to {backup_file}"}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/tables/{table_name}/restore", summary="Restore a
table from Excel")
def restore_table(table_name: str, request: TableRestoreRequest,
conn=Depends(get_db_connection), user: dict =
Depends(get_current_user)):
    if user['role'] != 'superuser':
        raise HTTPException(status_code=403, detail="Only
superusers can restore tables")
    try:
        df = pd.read_excel(request.file_path)
        with conn.cursor() as cursor:
            cursor.execute(f'TRUNCATE TABLE "{table_name}"
RESTART IDENTITY CASCADE')
            columns = ', '.join([f'"{col}"' for col in
df.columns])
            values_placeholder = ', '.join(['%s'] *
len(df.columns))
            insert_query = f'INSERT INTO "{table_name}"
({columns}) VALUES ({values_placeholder})'
            for _, row in df.iterrows():
                cursor.execute(insert_query, tuple(row))
            conn.commit()
            return {"message": f"Table {table_name} restored
successfully from {request.file_path}"}
    except Exception as e:
        conn.rollback()
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/database/backup", summary="Backup database")
def backup_database(user: dict = Depends(get_current_user)):
    # Allow backup for any user; operation is read-only
    backup_dir = "database_backups"
    if not os.path.exists(backup_dir):
        os.makedirs(backup_dir)

    timestamp = datetime.datetime.now().strftime("%Y%m%d%H%M%S")
    backup_file = os.path.join(backup_dir,
f"backup_{timestamp}.sql")

    try:
        command = [
            "pg_dump",
            "-U", DB_CONFIG["user"],
            "-h", DB_CONFIG["host"],

```

```

        "-p", str(DB_CONFIG["port"]),
        "-d", DB_CONFIG["database"],
        "-f", backup_file,
        "--format=p",
        "--no-owner",
        "--no-acl"
    ]
    env = os.environ.copy()
    env["PGPASSWORD"] = DB_CONFIG["password"]
    subprocess.run(command, check=True, env=env,
capture_output=True, text=True)
    return {"message": f"Backup created successfully at
{backup_file}"}
    except subprocess.CalledProcessError as e:
        raise HTTPException(status_code=500, detail=f"Backup
failed: {e.stderr}")
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))

@app.post("/database/restore", summary="Restore database from
SQL")
def restore_database(request: DatabaseRestoreRequest, user: dict
= Depends(get_current_user)):
    # Allow restore for any user (uses DB_CONFIG credentials
under the hood)
    try:
        if not os.path.exists(request.backup_file):
            raise HTTPException(status_code=400, detail=f"Backup
file not found: {request.backup_file}")

        temp_conn = psycopg2.connect(
            host=DB_CONFIG["host"],
            port=DB_CONFIG["port"],
            database='postgres',
            user=DB_CONFIG["user"],
            password=DB_CONFIG["password"]
        )
        temp_conn.autocommit = True
        temp_cursor = temp_conn.cursor()
        # Terminate any connections to the target DB before
dropping
        temp_cursor.execute(
            """
            SELECT pg_terminate_backend(pid)
            FROM pg_stat_activity
            WHERE datname = %s AND pid <> pg_backend_pid();
            """,
            (DB_CONFIG['database'],)
        )
        temp_cursor.execute(f"DROP DATABASE IF EXISTS
\"{DB_CONFIG['database']}\"")

```

```

        temp_cursor.execute(f"CREATE DATABASE
\"{DB_CONFIG['database']}\")
        temp_cursor.close()
        temp_conn.close()

    command = [
        "psql",
        "-U", DB_CONFIG["user"],
        "-h", DB_CONFIG["host"],
        "-p", str(DB_CONFIG["port"]),
        "-d", DB_CONFIG["database"],
        "-f", request.backup_file
    ]
    env = os.environ.copy()
    env["PGPASSWORD"] = DB_CONFIG["password"]
    subprocess.run(command, check=True, env=env,
capture_output=True, text=True)
    return {"message": "Database restored successfully."}
    except subprocess.CalledProcessError as e:
        raise HTTPException(status_code=500, detail=f"Restore
failed: {e.stderr}")
    except HTTPException as e:
        # pass-through for explicit HTTP errors above
        raise e
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Unexpected
error: {str(e)}")

@app.get("/queries", summary="Get saved queries")
def get_saved_queries(user: dict = Depends(get_user_or_guest)):
    try:
        if not os.path.exists("saved_queries.txt"):
            return {"queries": []}
        with open("saved_queries.txt", "r", encoding="utf-8") as
f:
            queries = [line.strip() for line in f.readlines()]
            return {"queries": queries}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/queries", summary="Save a query")
def save_query(query: QuerySave, user: dict =
Depends(get_user_or_guest)):
    try:
        with open("saved_queries.txt", "a", encoding="utf-8") as
f:
            f.write(query.query + '\n')
            return {"message": "Query saved successfully."}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
if __name__ == "__main__":
    uvicorn.run(app, host="127.0.0.1", port=8000)

```